```
*********************
    contador_sol.c
*********************

Apartado a)

Hilo 3077540720, contador = 0
Hilo 3077540720, contador = 1
Hilo 3077540720, contador = 3
Hilo 3077540720, contador = 6
Hilo principal, contador = 6

Nota: Aunque aparezca el mismo identificador para toods los hilos
subordinados (3077540720), realmente son hilos diferentes a los que
el sistema les asigna el mismo identificador. Fijemosnos que los hilos
subordinados no se ejecutan de forma concurrente en el codigo del programa.


Apartado b)

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NTHREADS 4

int  v[NTHREADS];

pthread_mutex_t mutex;
int contador=0;

void *hilo(void *arg)
{ int  i;

  i = *(int *) arg;

  pthread_mutex_lock(&mutex);
  contador = contador + i;
  printf("Hilo %lu, contador = %d\n", pthread_self(), contador);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}
int main()
{
  int i;
  pthread_t thread[NTHREADS]; //vector para definir los hilos

  pthread_mutex_init(&mutex, NULL);

  for (i=0;i<NTHREADS;i++)
  {
      v[i] = i;
      pthread_create(&thread[i], NULL, hilo, (void *)&v[i]);
  }
  for (i=0;i<NTHREADS;i++)
      pthread_join(thread[i],NULL);

  printf("Hilo principal, contador = %d\n",contador);
  pthread_mutex_destroy(&mutex);
  pthread_exit (0);
}


*********************
    suma_vector.c
*********************
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
```

```
#include <stdlib.h>
#include <time.h>
#define MAX 12

int v[MAX];
int n_hilos, dim_subvector;
int vsum[MAX];

int suma = 0;              // -> Cambio: A�adimos esta variable global

pthread_mutex_t mutex;

typedef struct {
   int iini;
   int ifin;
   int nhilo;
} parametros;

void *suma_hilo(void *arg)
{
   int i, ini, fin, nhilo; // -> Cambio: Quitamos suma como variable local
   parametros * argum;
   argum=(parametros *)arg;
   ini=argum->iini;
   fin=argum->ifin;
   nhilo=argum->nhilo;
   for (i=ini; i<=fin; i++)
   {
      pthread_mutex_lock(&mutex);   // -> Cambio
      suma = suma + v[i];
      pthread_mutex_unlock(&mutex); // -> Cambio
   }
   printf("Hilo %d (%lu): Suma entre componente %d y %d. Suma parcial: %d\n",
           nhilo,pthread_self(),ini,fin,suma);
   pthread_exit(NULL);
}
int main(int argc, char * argv[])
{
   time_t t;
   int i;
   pthread_t hilo[MAX];
   parametros * argum;

   if (argc==1)
   {   printf("Error: Has de introducir el n�mero de hilos\n");
      exit(-1);
   }

   n_hilos=atoi(argv[1]);
   dim_subvector=MAX / n_hilos;

   srandom(time(&t));

   pthread_mutex_init(&mutex, NULL);                    // -> Cambio

   printf("VECTOR\n");
   for (i=0;i<MAX;i++)
   {
      v[i]=random()%10;
      printf("V[%d]=%d\n",i,v[i]);
   }
   printf("\n");

   for (i=0;i<n_hilos-1;i++)
   {
      argum=(parametros *)malloc(sizeof(argum));
      argum->iini=i*dim_subvector;
      argum->ifin=((argum->iini) + dim_subvector -1);
      argum->nhilo=i;
```

```
      pthread_create(&hilo[i],NULL,&suma_hilo,(void *)argum);
   }
   /* Ultimo fragmento del vector */
   i=n_hilos - 1;
   argum=(parametros *)malloc(sizeof(argum));
   argum->iini=i*dim_subvector;
   argum->ifin=MAX-1;
   argum->nhilo=i;
   pthread_create(&hilo[i],NULL,&suma_hilo,(void *)argum);

   for (i=0;i<n_hilos;i++) pthread_join(hilo[i],NULL); // -> Cambio

   pthread_mutex_destroy(&mutex);                      // -> Cambio

   printf("Suma de todas las componentes del vector: %d\n",suma);
   pthread_exit(0);
}


**********************
    suma_vector2.c
**********************

#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
#define MAX 12

int v[MAX];
int n_hilos, dim_subvector;
int vsum[MAX];

int suma = 0;

pthread_mutex_t mutex;

typedef struct {
   int iini;
   int ifin;
   int nhilo;
} parametros;

void *suma_hilo(void *arg)
{
   int i, ini, fin, nhilo, suma_hilo=0;   // -> Cambio respecto a suma_vector.c
   parametros * argum;
   argum=(parametros *)arg;
   ini=argum->iini;
   fin=argum->ifin;
   nhilo=argum->nhilo;
   for (i=ini; i<=fin; i++)
      suma_hilo = suma_hilo + v[i];     // -> Cambio: Es m�s eficiente acumular sobre una
variable local

   pthread_mutex_lock(&mutex);
   suma = suma + suma_hilo;             // -> Cambio: A�ado la suma calculada por el hilo a la
varible global
   pthread_mutex_unlock(&mutex);

   printf("Hilo %d (%lu): Suma entre componente %d y %d. Suma parcial: %d\n",
           nhilo,pthread_self(),ini,fin,suma);
   pthread_exit(NULL);
}
int main(int argc, char * argv[])
{
   time_t t;
   int i;
```

```
   pthread_t hilo[MAX];
   parametros * argum;

   if (argc==1)
   {   printf("Error: Has de introducir el n◆mero de hilos\n");
      exit(-1);
   }

   n_hilos=atoi(argv[1]);
   dim_subvector=MAX / n_hilos;

   srandom(time(&t));

   pthread_mutex_init(&mutex, NULL);

   printf("VECTOR\n");
   for (i=0;i<MAX;i++)
   {
      v[i]=random()%10;
      printf("V[%d]=%d\n",i,v[i]);
   }
   printf("\n");

   for (i=0;i<n_hilos-1;i++)
   {
      argum=(parametros *)malloc(sizeof(argum));
      argum->iini=i*dim_subvector;
      argum->ifin=((argum->iini) + dim_subvector -1);
      argum->nhilo=i;
      pthread_create(&hilo[i],NULL,&suma_hilo,(void *)argum);
   }
   /* Ultimo fragmento del vector */
   i=n_hilos - 1;
   argum=(parametros *)malloc(sizeof(argum));
   argum->iini=i*dim_subvector;
   argum->ifin=MAX-1;
   argum->nhilo=i;
   pthread_create(&hilo[i],NULL,&suma_hilo,(void *)argum);

   for (i=0;i<n_hilos;i++) pthread_join(hilo[i],NULL);

   pthread_mutex_destroy(&mutex);

   printf("Suma de todas las componentes del vector: %d\n",suma);
   pthread_exit(0);
}


**********************
    imprime_matriz_sol.c
**********************
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#define MAX_DIM 10
int n=4;
typedef int TMat [MAX_DIM][MAX_DIM];
TMat A={{1,2,5,1},{2,3,8,0},{3,8,5,1},{4,0,4,1}};
int turno=0;
pthread_mutex_t mutex;
pthread_cond_t  TocaTurnoFila;

void *Impr (void *arg)
{ int i;
  int f;

  f=*(int *)arg;
  pthread_mutex_lock(&mutex);
```

```
  while ( turno != f )
     pthread_cond_wait(&TocaTurnoFila,&mutex);
  printf("Fila %d (Hilo %lu):  ", f, pthread_self());
  for (i=0; i<n; i++) printf("%d  ", A[f][i]);
  printf("\n");
  turno++;
  pthread_cond_broadcast (&TocaTurnoFila);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}
int main()
{
  int i,j;
  int v[MAX_DIM];
  pthread_t hilo[MAX_DIM];

  pthread_mutex_init(&mutex, NULL);
  pthread_cond_init(&TocaTurnoFila,NULL);

  printf("Matriz impresa por el hilo principal\n");
  for (i=0; i<n; i++)
  { printf("Fila %d:  ", i);
    for (j=0; j<n; j++) printf("%d  ", A[i][j]);
    printf("\n");
  }

  printf("Matriz impresa por los hilos subordinados\n");

  for (i=0; i<n; i++)
  { v[i]=i;
    pthread_create(&hilo[i], NULL,  Impr, (void *)&v[i]);
  }
  for (i=0; i<n; i++) pthread_join(hilo[i], NULL);

  pthread_cond_destroy(&TocaTurnoFila);
  pthread_mutex_destroy(&mutex);

  exit(0);
}

**********************
    exam_ene2013_sol.c
**********************
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

#define NHILOS  5


pthread_mutex_t m;
pthread_cond_t toca_turno;

int turno=0;

void *f_hilos(void *ind)
{
 int indice;

 indice = *(int *)ind;

 while(1) {
      pthread_mutex_lock(&m);
      while(turno != indice)
           pthread_cond_wait(&toca_turno, &m);
      turno = (indice + 1) % NHILOS;
      pthread_cond_broadcast(&toca_turno);
      printf("Comensal %d: Sirve comida\n",indice);
```

```
      printf("Comensal %d: Comiendo\n",indice);
      pthread_mutex_unlock(&m);

      sleep(random() % 3); /* Comensal ind comiendo */

      pthread_mutex_lock(&m);
      printf("Comensal %d: Plato vac�o\n",indice);
      pthread_mutex_unlock(&m);
 }
}

int main()
{
 int i, size, datum, *v;
 pthread_t *hilos;
 time_t t;

 srandom(time(&t));

 hilos = malloc(NHILOS * sizeof(pthread_t));
 v     = malloc(NHILOS * sizeof(v));

 pthread_mutex_init(&m, NULL);
 pthread_cond_init(&toca_turno, NULL);

 for (i = 0; i < NHILOS; i++)
 {   v[i] = i;
     pthread_create(&hilos[i], NULL, f_hilos, (void *)&v[i]);
 }
 for (i = 0; i < NHILOS; i++) pthread_join(hilos[i], NULL);

 pthread_mutex_destroy(&m);
 pthread_cond_destroy(&toca_turno);
}


**********************
    cores.c
**********************
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

#define N_CORES 8

int num_hilos, fd, turno=0;
pthread_mutex_t mutex;
pthread_cond_t *toca_turno;

int ejecuta (int hilo, int n_core)
{  int resultado = random()%2;
   sleep(resultado);
   return resultado;
}

void escribe_en_fichero (int iteracion, int n_core, int resultado)
{     write(fd, &iteracion, sizeof(int));
      write(fd, &n_core, sizeof(int));
      write(fd, &resultado, sizeof(int));
      printf("%d %d %d \n", iteracion, n_core, resultado);
}

void *f_hilo(void *indice)
{ int i, n_core, resultado;
  n_core = *(int *)indice;
```

```
  for (i=0; i<10; i++)
  {   resultado = ejecuta (i, n_core);

      pthread_mutex_lock(&mutex);
      while(turno != n_core)
           pthread_cond_wait(&toca_turno[n_core], &mutex);

      escribe_en_fichero  (i, n_core, resultado);

      turno = (n_core+ 1) % N_CORES;
      pthread_cond_signal(&toca_turno[turno]);

      pthread_mutex_unlock(&mutex);
  }
  pthread_exit(0);
}

int main()
{ int i, tamanyo, dato;
  pthread_t *mis_hilos;
  int *v;

  time_t t;
  srandom(time(&t));

  fd=open("fichero",O_WRONLY);

  num_hilos = N_CORES;

  v = malloc(num_hilos * sizeof(int));

  mis_hilos = malloc(num_hilos * sizeof(pthread_t));

  toca_turno = malloc(num_hilos * sizeof(pthread_cond_t));

  pthread_mutex_init(&mutex, NULL);

  for (i = 0; i < num_hilos; i++)
        pthread_cond_init(&toca_turno[i], NULL);

  for (i = 0; i < num_hilos; i++)
  {
      v[i] = i;
      pthread_create(&mis_hilos[i], NULL, f_hilo, (void *)&v[i]);
  }
  for (i = 0; i < num_hilos; i++)
      pthread_join(mis_hilos[i], NULL);

  pthread_mutex_destroy(&mutex);

  for (i = 0; i < num_hilos; i++)
        pthread_cond_destroy(&toca_turno[i]);

  close(fd);

  pthread_exit(0);
}


**********************
    tabla_mult.c
**********************

/* tabla_mult.c imprime las 10 tablas de multiplicar usando
   una ÚNICA VARIABLE DE CONDICIÓN. Es una solución viable
   pero poco eficiente porque avisa a todos los hilos
   bloqueados en dicha variable de condición y algunos de
   esos hilos no deberían desbloquearse y han de bloquearse de nuevo */
```

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define DIM 10

pthread_mutex_t mutex;
pthread_cond_t  turno_imprimir;
int turno_tabla=1;
int v_indice[DIM+1];

// En el programa usamos vecotres de DIM+1 componentes porque en C los
// componentes de un vector empiezan a numerarse por 0. Pero trataremos con
// las componentes 1 hasta 10 de esos vectores para que la implementaci□n
// resulte m□s sencilla y legible.

void *f_tabla(void *arg)
{
  int i;
  int n_tabla;
  n_tabla=*(int *)arg;

  for (i=1; i<(DIM+1); i++)
  {  pthread_mutex_lock(&mutex);
     while ( turno_tabla != n_tabla)
        pthread_cond_wait(&turno_imprimir,&mutex);

     printf("%d x %d = %d  \t",n_tabla,i,n_tabla*i);
     turno_tabla++;
     if (turno_tabla == (DIM+1)) { turno_tabla=1; printf("\n");}

     pthread_cond_broadcast(&turno_imprimir);
     pthread_mutex_unlock(&mutex);
  }
  pthread_exit(0);
}

int main()
{
  int i;
  pthread_t hilo[DIM+1];

  pthread_mutex_init(&mutex, NULL);
  pthread_cond_init(&turno_imprimir,NULL);

  for (i=1; i<(DIM+1); i++)
  { v_indice[i]=i;
    pthread_create(&hilo[i], NULL,  f_tabla, (void *)&v_indice[i]);
  }
  for (i=1; i<(DIM+1); i++) pthread_join(hilo[i], NULL);

  pthread_cond_destroy(&turno_imprimir);
  pthread_mutex_destroy(&mutex);

  pthread_exit(0);
}



**********
    tabla_mult2.c
**********

/* tabla_mult2.c imprime las 10 tablas de multiplicar usando
   un VECOTR DE VARIABLES DE CONDICI□N. Esta implementaci□n
   avisa (desbloqea) directamente al hilo que debe imprimir
   a continuaci□n su parte de la tabla.  */
```

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>


#define DIM 10

pthread_mutex_t mutex;
pthread_cond_t  turno_imprimir[DIM+1];
int turno_tabla=1;

void *f_tabla(void *arg)
{
  int i;
  int n_tabla;
  n_tabla=*(int *)arg;

  for (i=1; i<(DIM+1); i++)
  {  pthread_mutex_lock(&mutex);
     while ( turno_tabla != n_tabla)
        pthread_cond_wait(&turno_imprimir[n_tabla],&mutex);

     printf("%d x %d = %d  \t",n_tabla,i,n_tabla*i);
     turno_tabla++;
     if (turno_tabla == (DIM+1)) { turno_tabla=1; printf("\n");}

     pthread_cond_signal(&turno_imprimir[turno_tabla]);
     pthread_mutex_unlock(&mutex);
  }
  pthread_exit(0);
}

int main()
{
  int i, v[DIM];
  pthread_t hilo[DIM+1];

  pthread_mutex_init(&mutex, NULL);
  for (i=1; i<(DIM+1); i++) pthread_cond_init(&turno_imprimir[i],NULL);


  for (i=1; i<(DIM+1); i++)
  {
     v[i] = i;
     pthread_create(&hilo[i], NULL,  f_tabla, (void *)&v[i]);
  }
  for (i=1; i<(DIM+1); i++) pthread_join(hilo[i], NULL);

  for (i=1; i<(DIM+1); i++) pthread_cond_destroy(&turno_imprimir[i]);
  pthread_mutex_destroy(&mutex);

  pthread_exit(0);
}



****************************************
    ejercicio2_traspas_con_1_var_cond.c
****************************************
#include <pthread.h>
#include <stdio.h>

#define MAX 20

pthread_mutex_t mutex;
pthread_cond_t c1;
int turno;
```

```
void *GeneraPares(void *arg){
  int i;
  for (i=0; i<MAX; i=i+2){
    pthread_mutex_lock(&mutex);
    while(turno==1)
      pthread_cond_wait(&c1,&mutex);
    printf("(Hilo %lu): %d\n",pthread_self(),i);
    turno=1;
    pthread_cond_signal(&c1);
    pthread_mutex_unlock(&mutex);
  }
  pthread_exit(0);
}

void *GeneraImpares(void *arg){
  int i;
  for (i=1; i<MAX; i=i+2){
    pthread_mutex_lock(&mutex);
    while (turno==0)
      pthread_cond_wait(&c1,&mutex);
    printf("(Hilo %lu): %d\n",pthread_self(),i);
    turno=0;
    pthread_cond_signal(&c1);
    pthread_mutex_unlock(&mutex);
  }
  pthread_exit(0);
}

int main(){
  pthread_t Pares, Impares;
  pthread_mutex_init(&mutex,NULL);
  pthread_cond_init(&c1,NULL);
  turno=0;
  pthread_create(&Pares, NULL, &GeneraPares, NULL);
  pthread_create(&Impares, NULL, &GeneraImpares, NULL);
  pthread_join(Pares,NULL);
  pthread_join(Impares,NULL);
  pthread_cond_destroy(&c1);
  pthread_mutex_destroy(&mutex);
}
/* Si se sustituye
      for (i=0; i<MAX; i=i+2)
   por
      for (i=2; i<MAX; i=i+2)
   en el hilo GeneraPares el programa no finaliza. Podr as explicar por qu ?
*/


****************************************
    examen_noviembre_2013 Pregunta 1
****************************************
En este c digo el hilo thread_A lee datos desde teclado y los deja en un buffer.
El hilo  thread_B coge uno a uno los datos del buffer y los muestra por pantalla.
El buffer nunca se llenar  y, por tanto,  hread_A nunca se bloquear  por no tener
espacio en el buffer para almacenar un dato le do. Pero  thread_B s  que debe
bloquearse cuando el buffer  no contenga ning n elemento. La variable "valor", accesible por
ambos hilos, contabiliza el n mero de elementos de buffer que no han sido le dos
por  thread_B. Por tanto,  thread_B deber  bloquearse en una variable
de condici n mientras "valor" valga 0.

El problema a resolver se asemeja al del productor-consumidor con
buffer infinito, en el que el productor  nunca se bloquea ya que el buffer no se llena
nunca.

Por otro lado, el acceso al buffer compartido y a la variable global "valor" deber
realizarse dentro del mutex. La instrucci n "scanf" no es preciso ejecutarla dentro
del mutex.  Y la instrucci n "printf" tampoco, ya que solo un hilo puede imprimir
a la vez y, consecuentemente, no se mezclar n nunca varios mensajes.
```

Un error ser�a solucionar el problema usando una variable de condici�n para
establecer turnos alternados entre los dos hilos. El hilo A y B se deben ejecutar
de forma concurrente sin ning�n tipo de sincronizaci�n, excepto lo comentado anteriormente.

Teniendo todo esto en cuenta, una posible implementaci�n que resuelve el problema ser�a la
siguiente:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#define MAXBUF 10
int BUF[MAXBUF];
int valor=0;
pthread_mutex_t m;
pthread_cond_t c;

void *A(void *arg)
{
  int i, j;
 for (i=0; i<MAXBUF; i++){
   scanf("%d",&j);
   fflush(stdin);
   pthread_mutex_lock(&m);
   BUF[i]=j;
   valor++;
   pthread_cond_signal(&c);
   pthread_mutex_unlock(&m);
 }
 pthread_exit(0);
}

void *B(void *arg)
{
  int i, j;

  for (i=0; i<MAXBUF; i++){
    pthread_mutex_lock(&m);
    while(valor==0)
      pthread_cond_wait(&c,&m);
    j=BUF[i];
    valor--;
    pthread_mutex_unlock(&m);
    printf("Leido %d\n",j);
  }
  pthread_exit(0);
}

int main(){
    int v=0;
    pthread_t thread_A,thread_B;
    pthread_create(&thread_A, NULL, A, NULL);
    pthread_create(&thread_B, NULL, B, NULL);
    pthread_join(thread_A,NULL);
    pthread_join(thread_B,NULL);
}
```

****
    examen_noviembre_2013 Pregunta 2
****
    a) Falso. El hilo thread_A tiene acceso a sus variables locales y a las
       variables globales accesibles por todos los hilos como "valor". Pero no tiene acceso a las
       variables locales de otro hilo, como "v".
    b) Falso. Cuando se crea un hilo este ha de ejecutar la funci�n que se especifica
       en el tercer argumento de "pthread_create". Por tanto, el contador de programa
       de dicho hilo apuntar� a la primera instrucci�n de esta funci�n.

c) Cierto. Cada hilo tiene su propio entorno de ejecuci□n y, consecuentemente, su propia pila de ejecuci□n en la que almacenar dicho entorno.

```
****************************************
    examen_noviembre_2013 Pregunta 3
****************************************
```

Asumiremos que tenemos una m□quina con un □nico procesador.

a) Si el hilo creado se asigna al mismo procesador que el hilo que lo crea, el que ejecuta "pthread_create", podr□a continuar en ejecuci□n o, si el turno del procesador pasase al hilo creado, podr□a pasar al estado listo. No hay que comentar nada respecto al hilo creado porque el enunciado pregunta por los cambios producidos en el hilo que ejecuta "pthread_create.

b) La funci□n "pthread_self", como □nicamente obtiene el n□mero de identificaci□n del hilo que lo ejecuta, no provoca ning□n cambio de estado en dicho hilo.

c) La funci□n "pthread_cond_wait" bloquea de forma incondicional al hilo que la ejecuta. Por tanto, el hilo cambia del estado de ejecuci□n al de bloqueado.

d) Un hilo ejecuta esta funci□n para esperar a que otro finalice. Si este □ltimo hilo no ha finalizado, el hilo que ejecuta "pthread_join" se bloquea. Consecuentemente, cambiar□ su estado de ejecuci□n a bloqueado. Si el hilo por el que pretend□a esperar ya ha finalizado, el hilo que ejecuta "pthread_join" continuar□ en estado de ejecuci□n.

   Algunos de vosotros indic□bais que, cuando el hilo que invoca "pthread_join" sale de la situaci□n de bloqueo, ejecuta "pthread_exit" y finaliza. Esto no ha de ocurrir as□ necesariamente.

   Y tampoco es cierto que "pthread_join" lo ejecute siempre el hilo principal para esperar por uno de los hilos que ha creado. La funci□n "pthread_join", a diferencia de la funci□n wait para procesos, puede ejecutarla cualquier hilo para esperar a que finalice cualquier otro hilo.

```
*************************
    examen_enero_2014
*************************
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define N_HILOS  10

int n_lectores, n_escritores;
pthread_mutex_t mutex;
pthread_cond_t se_puede_acceder;

/* La implementaci□n de estas funciones se puede mejorar */

void *f_lector (void *indice)
{ int i, n_lector, resultado;
  n_lector = *(int *)indice;
  sleep(random()%5);
  printf("Lector %d llega\n",n_lector);

  pthread_mutex_lock(&mutex);
  while (n_escritores == 1)
  {
           printf("Lector %d espera\n",n_lector);
           pthread_cond_wait(&se_puede_acceder, &mutex);
  }
  n_lectores++;
  printf("Lector %d va a leer\n",n_lector);
```

```
  pthread_mutex_unlock(&mutex);

  sleep(random()%3); /* Simula acceso lectura fichero */

  pthread_mutex_lock(&mutex);
  printf("Lector %d deja de leer\n",n_lector);
  n_lectores--;
  if (n_lectores == 0) pthread_cond_signal(&se_puede_acceder);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}

void *f_escritor (void *indice)
{ int i, n_escritor, resultado;
  n_escritor=  *(int *)indice;
  sleep(random()%3);
  printf("Escritor %d llega\n",n_escritor);

  pthread_mutex_lock(&mutex);
  while ((n_escritores == 1) || (n_lectores >= 1))
  {
          printf("Escritor %d espera\n",n_escritor);
          pthread_cond_wait(&se_puede_acceder, &mutex);
  }
  n_escritores++;
  printf("Escritor %d va a escribir\n",n_escritor);

  sleep(random()%3); /* Simula acceso lectura fichero */

  printf("Escritor %d deja de escribir\n",n_escritor);
  n_escritores--;
  pthread_cond_broadcast(&se_puede_acceder);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}

int main()
{ int i, *v;
  pthread_t *hilos_lectores, *hilos_escritores;

  time_t t;
  srandom(time(&t));

  n_lectores =   0;
  n_escritores = 0;

  hilos_lectores   = malloc(N_HILOS * sizeof(pthread_t));
  hilos_escritores = malloc(N_HILOS * sizeof(pthread_t));

  v                = malloc(N_HILOS * sizeof(int));

  pthread_mutex_init(&mutex, NULL);
  pthread_cond_init(&se_puede_acceder, NULL);

  for (i = 0; i < N_HILOS; i++)
  {   v[i] = i;
      pthread_create(&hilos_lectores[i], NULL, f_lector, (void *)&v[i]);
  }
  for (i = 0; i < N_HILOS; i++)
      pthread_create(&hilos_escritores[i], NULL, f_escritor, (void *)&v[i]);
  for (i = 0; i < N_HILOS; i++)
      pthread_join(hilos_lectores[i], NULL);
  for (i = 0; i < N_HILOS; i++)
      pthread_join(hilos_escritores[i], NULL);

  pthread_mutex_destroy(&mutex);
  pthread_cond_destroy(&se_puede_acceder);

  pthread_exit(0);
```

```
}


************************
   acceso_a_datos_2.c
************************
//  Ejercicio 3 de las traspas con la restriccion adicional de que como maximo pueden
//  acceder 5 procesos del mismo tipo
//
#include <pthread.h>
#include <stdio.h>
#include <time.h>

#define MaxA   10
#define MaxB   10

int nA=0, nB=0, n=0;
pthread_mutex_t m;
pthread_cond_t  noprocsA, noprocsB;

void *f_hiloA(void *arg);
void *f_hiloB(void *arg);


int main()
{
 pthread_t hiloA[MaxA], hiloB[MaxB];
 int i;
 time_t t;

 srandom(time(&t));

 pthread_mutex_init(&m, NULL);
 pthread_cond_init(&noprocsA,NULL);
 pthread_cond_init(&noprocsB,NULL);

 for (i=0; i<MaxA; i++)
    pthread_create(&hiloA[i], NULL, &f_hiloA, NULL);
 for (i=0; i<MaxB; i++)
    pthread_create(&hiloB[i], NULL, &f_hiloB, NULL);

 for (i=0; i<MaxA; i++) pthread_join(hiloA[i], NULL);
 for (i=0; i<MaxB; i++) pthread_join(hiloB[i], NULL);

 pthread_cond_destroy(&noprocsA);
 pthread_cond_destroy(&noprocsB);
 pthread_mutex_destroy(&m);
 pthread_exit(0);
}

void *f_hiloA (void *arg)
{

 sleep(random()%5);

 pthread_mutex_lock(&m);
 while ((nB > 0) || (nA == 5))
 {
    printf("(A: %d %d %lu) Espero\n",nA,nB,pthread_self());
    pthread_cond_wait(&noprocsB,&m);
 }
 nA++;
 n++;
 pthread_mutex_unlock(&m);

 /*Acceso a datos */
 printf("(A: %d %d %lu) Accedo a datos\n",nA,nB,pthread_self());
 sleep(random()%3);
 printf("(A: %d %d %lu) Dejo acceso a datos\n",nA,nB,pthread_self());
```

```
 /* Fin acceso datos */

 pthread_mutex_lock(&m);
 nA--;
 if ((nA>0) && (nA <= 4)) pthread_cond_broadcast(&noprocsB);
 if (nA == 0)  // Ojo! No vale poner else if
   {pthread_cond_broadcast(&noprocsA);
    printf("\n");}
 pthread_mutex_unlock(&m);

 pthread_exit(0);
}


void *f_hiloB (void *arg)
{

 sleep(random()%5);

 pthread_mutex_lock(&m);
 while ((nA > 0) || (nB == 5))
 {
    printf("(B: %d %d %lu) Espero\n",nA,nB,pthread_self());
    pthread_cond_wait(&noprocsA,&m);
 }
 nB++;
 n++;
 pthread_mutex_unlock(&m);

 /*Acceso a datos */
 printf("(B: %d %d %lu) Accedo a datos\n",nA,nB,pthread_self());
 sleep(random()%3);
 printf("(B: %d %d %lu) Dejo acceso a datos\n",nA,nB,pthread_self());
 /* Fin acceso datos */

 pthread_mutex_lock(&m);
 nB--;
 if ((nB>0) && (nB <= 4)) pthread_cond_broadcast(&noprocsA);
 if (nB == 0)
   {pthread_cond_broadcast(&noprocsB);
    printf("\n");}

 pthread_mutex_unlock(&m);

 pthread_exit(0);
}
```

****
Examen noviembre 2014
****

Para empezar ambos hilos han de acceder a la variable global v dentro de un mutex.
El hilo restador, si el valor de la variable v es cero, se tiene que esperar a que
esa variable no sea nula antes de decrementarla ya que, si no, tendr�a un valor negativo. Por otro lado,
el hilo sumador, no tiene problemas para incrementar la variable v. �nicamente ha
de tener en cuenta que, si tras el incremento la variable vale uno, hay que avisar
a los hilos restadores por si estuvisen bloqueados esperando a que la variable v no fuese nula.

N�tese que, si cambiamos la funci�n pthread_cond_broadcast por pthread_cond_signal,
podr�a darse el caso de que quedasen hilos restadores bloqueados cuando hubiesen acabado todos los hilos sumadores. Por ejemplo, si empezaran a ejecutarse todos los hilos restadores
antes que los sumadores.

Teniendo esto en cuenta, una posible implementaci�n del programa ser�a la siguiente:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
#define numthreads 10

int v=0;

pthread_mutex_t m;
pthread_cond_t v_no_nula;

void *sumador(void *index)
{
 int i, ind;

 ind = (int)index;

 for (i=0; i<1000; i++)
 {
      pthread_mutex_lock(&m);
      v++;
      printf("Hilo %d iteracion %d suma: %d\n", ind, i, v);
      if (v==1) pthread_cond_broadcast(&v_no_nula);
      pthread_mutex_unlock(&m);
 }
 pthread_exit(0);
}

void *restador(void *index)
{
 int i, ind;

 ind = (int)index;

 for (i=0; i<1000; i++)
 {
      pthread_mutex_lock(&m);
      while(v== 0) pthread_cond_wait(&v_no_nula, &m);
      v--;
      printf("Hilo %d iteracion %d resta: %d\n", ind, i, v);
      pthread_mutex_unlock(&m);
 }
 pthread_exit(0);
}

int main(int argc, char *argv[])
{
 pthread_t *hilo_sumador, *hilo_restador;
 int i, *v;

 hilo_sumador  =  malloc(numthreads * sizeof(pthread_t));
 hilo_restador = malloc(numthreads * sizeof(pthread_t));
 v             = malloc(numthreads * sizeof(int));

 pthread_mutex_init(&m, NULL);
 pthread_cond_init(&v_no_nula, NULL);

 for (i = 0; i < numthreads; i++)
 {   v[i] = i;
     pthread_create(&hilo_sumador[i], NULL, sumador, (void *)&v[i]);
 }
 for (i = 0; i < numthreads; i++)
     pthread_create(&hilo_restador[i], NULL, restador, (void *)&v[i]);

 for (i = 0; i < numthreads; i++)
     pthread_join(hilo_sumador[i], NULL);
 for (i = 0; i < numthreads; i++)
     pthread_join(hilo_restador[i], NULL);

 pthread_exit(0);
}
```

****
Examen enero 2015
****
El planteamiento de este ejercicio se ajusta a un esquema de turnos entre dos tipos de hilos similar al
ejercicio que vimos en clase en el que un hilo imprim�a los n�meros pares y otro, los impares.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define numthreads 10

int v=0;
int sumar=1;

pthread_mutex_t m;
pthread_cond_t continuar_sumador, continuar_restador;

void *sumador(void *index)
{
 int i, ind;

 ind = *(int *)index;

 for (i=0; i<1000; i++)
 {
      pthread_mutex_lock(&m);
      while (!sumar)
        pthread_cond_wait(&continuar_sumador,&m);
      v++;
      printf("Hilo %d (%lu) iteraci�n %d suma: %d\n", ind, pthread_self(), i, v);
      sumar=0;
      pthread_cond_signal(&continuar_restador);
      pthread_mutex_unlock(&m);
 }
 pthread_exit(0);
}

void *restador(void *index)
{
 int i, ind;

 ind = *(int *)index;

 for (i=0; i<1000; i++)
 {
      pthread_mutex_lock(&m);
      while (sumar)
         pthread_cond_wait(&continuar_restador,&m);
      v--;
      printf("Hilo %d (%lu) iteraci�n %d resta: %d\n", ind, pthread_self(), i, v);
      sumar=1;
      pthread_cond_signal(&continuar_sumador);
      pthread_mutex_unlock(&m);
 }
 pthread_exit(0);
}


int main()
{
 pthread_t *hilo_sumador, *hilo_restador;
 int i, *v;

 if ((hilo_sumador= malloc(numthreads * sizeof(pthread_t))) == NULL) {
    perror("Cannot create pthread_t vector");
    exit(-1);
 }
```

```
 if ((hilo_restador= malloc(numthreads * sizeof(pthread_t))) == NULL) {
    perror("Cannot create pthread_t vector");
    exit(-1);
 }

 if ((v = malloc(numthreads * sizeof(int))) == NULL) {
    perror("Cannot create v vector");
    exit(-1);
 }

 if (pthread_mutex_init(&m, NULL)) {
      perror("Cannot init mutex");
      exit(-1);
 }
 pthread_cond_init(&continuar_sumador,NULL);
 pthread_cond_init(&continuar_restador,NULL);

 for (i = 0; i < numthreads; i++) v[i] = i;

 for (i = 0; i < numthreads; i++)
     if  (pthread_create(&hilo_sumador[i], NULL, sumador, (void *)&v[i])) {
         printf("On iteration %d: ", i);
         perror("error creating thread sumador");
         exit(-1);
     }
 for (i = 0; i < numthreads; i++)
     if  (pthread_create(&hilo_restador[i], NULL, restador, (void *)&v[i])) {
         printf("On iteration %d: ", i);
         perror("error creating thread restador");
         exit(-1);
     }
 printf("\n%d threads successfully created.\n", i);

 for (i = 0; i < numthreads; i++) {
     if  (pthread_join(hilo_sumador[i], NULL)) {
         printf("On iteration %d: ", i);
         perror("error joining thread sumador");
         exit(-1);
     }
 }
 for (i = 0; i < numthreads; i++) {
     if  (pthread_join(hilo_restador[i], NULL)) {
         printf("On iteration %d: ", i);
         perror("error joining thread sumador");
         exit(-1);
     }
 }
 printf("\n%d threads successfully joined.\n", i);

 pthread_cond_destroy(&continuar_sumador);
 pthread_cond_destroy(&continuar_restador);

 pthread_mutex_destroy(&m);

 pthread_exit(0);
}
```

Si se reemplaza en este c�digo la funci�n pthread_cond_signal por pthread_cond_broadcast la implementaci�n ser�a menos eficiente ya que, aunque se desbloqueen varios hilos de la variable de condici�n asociada a la funci�n invocada, solo uno de ellos podr� continuar su ejecuci�n. El resto han de bloquearse de nuevo.

A continuaci�n se propone una soluci�n alternativa que utiliza una �nica variable de condici�n. No obstante, esta implementaci�n es bastante menos eficiente que la anterior ya que la funci�n pthread_cond_broadcast que hay que utilizar desbloquea a todos los hilos bloqueados en la variable de condici�n y solo los hilos del tipo distinto al que realiz� la �ltima operaci�n pueden continuar

realmente la ejecuci&#65533;n.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define numthreads 10

int v=0;
int sumar=1;

pthread_mutex_t m;
pthread_cond_t continuar;

void *sumador(void *index)
{
 int i, ind;

 ind = *(int *)index;

 for (i=0; i<1000; i++)
 {
      pthread_mutex_lock(&m);
      while (!sumar)
        pthread_cond_wait(&continuar,&m);
      v++;
      printf("Hilo %d (%lu) iteraci&#65533;n %d suma: %d\n", ind, pthread_self(), i, v);
      sumar=0;
      pthread_cond_broadcast(&continuar);
      pthread_mutex_unlock(&m);
 }
 pthread_exit(0);
}

void *restador(void *index)
{
 int i, ind;

 ind = *(int *)index;

 for (i=0; i<1000; i++)
 {
      pthread_mutex_lock(&m);
      while (sumar)
         pthread_cond_wait(&continuar,&m);
      v--;
      printf("Hilo %d (%lu) iteraci&#65533;n %d resta: %d\n", ind, pthread_self(), i, v);
      sumar=1;
      pthread_cond_broadcast(&continuar);
      pthread_mutex_unlock(&m);
 }
 pthread_exit(0);
}


int main()
{
 pthread_t *hilo_sumador, *hilo_restador;
 int i, *v;

 if ((hilo_sumador= malloc(numthreads * sizeof(pthread_t))) == NULL) {
    perror("Cannot create pthread_t vector");
    exit(-1);
 }

 if ((hilo_restador= malloc(numthreads * sizeof(pthread_t))) == NULL) {
    perror("Cannot create pthread_t vector");
    exit(-1);
 }
```

```
 if ((v = malloc(numthreads * sizeof(int))) == NULL) {
    perror("Cannot create v vector");
    exit(-1);
 }

 if (pthread_mutex_init(&m, NULL)) {
      perror("Cannot init mutex");
      exit(-1);
 }
 pthread_cond_init(&continuar,NULL);

 for (i = 0; i < numthreads; i++)  v[i] = i;

 for (i = 0; i < numthreads; i++)
     if  (pthread_create(&hilo_sumador[i], NULL, sumador, (void *)&v[i])) {
         printf("On iteration %d: ", i);
         perror("error creating thread sumador");
         exit(-1);
     }
 for (i = 0; i < numthreads; i++)
     if  (pthread_create(&hilo_restador[i], NULL, restador, (void *)&v[i])) {
         printf("On iteration %d: ", i);
         perror("error creating thread restador");
         exit(-1);
     }
 printf("\n%d threads successfully created.\n", i);

 for (i = 0; i < numthreads; i++) {
     if  (pthread_join(hilo_sumador[i], NULL)) {
         printf("On iteration %d: ", i);
         perror("error joining thread sumador");
         exit(-1);
     }
 }
 for (i = 0; i < numthreads; i++) {
     if  (pthread_join(hilo_restador[i], NULL)) {
         printf("On iteration %d: ", i);
         perror("error joining thread sumador");
         exit(-1);
     }
 }
 printf("\n%d threads successfully joined.\n", i);

 pthread_cond_destroy(&continuar);

 pthread_mutex_destroy(&m);

 pthread_exit(0);
}
```

****
Examen noviembre 2015
****

a) La variable n es una variable local al hilo principal. Por lo tanto,
   solo este hilo tiene acceso a ella. Sin embargo, la variable v es
   una variable global y, consecuentemente, es accesible por todos los
   hilos del programa.

b) Si solo puede haber un hilo bloqueado en la variable de condici�n asociada, s�.
   En caso contrario podr�a ocurrir que la funci�n pthread_cond_signal
   desbloquee a un hilo que no cumpla la condici�n asociada a la funci�n pthread_cond_wait,
   con lo que vuelve a bloquearse. Si alg�n hilo bloqueado en la variable de condici�n
   cumple esa condici�n asociada a pthread_cond_wait y nadie lo desbloquea, el
   programa podr�a no finalizar nunca.

****
Examen noviembre 2015
****

Soluci&#65533;n I (con una variable de condici&#65533;n):

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define N_HILOS  5

int n_lectores=0, n_escritores=0, n_escritores_esperando=0;
pthread_mutex_t mutex;
pthread_cond_t se_puede_acceder;

void *f_lector (void *indice)
{ int i, n_lector, resultado;
  n_lector = *(int *)indice;
  sleep(random()%5);
  printf("Lector %d llega\n",n_lector);

  pthread_mutex_lock(&mutex);
  while ((n_escritores == 1) || (n_escritores_esperando>0))
  {
           printf("Lector %d espera\n",n_lector);
           pthread_cond_wait(&se_puede_acceder, &mutex);
  }
  n_lectores++;
  printf("Lector %d va a leer\n",n_lector);
  pthread_mutex_unlock(&mutex);

  sleep(random()%3); /* Simula acceso lectura fichero */

  pthread_mutex_lock(&mutex);
  printf("Lector %d deja de leer\n",n_lector);
  n_lectores--;
  if (n_escritores_esperando>0)  pthread_cond_broadcast(&se_puede_acceder);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}

void *f_escritor (void *indice)
{ int i, n_escritor, resultado;
  n_escritor=  *(int *)indice;
  sleep(random()%3);
  printf("Escritor %d llega\n",n_escritor);

  pthread_mutex_lock(&mutex);
  while ((n_escritores == 1) || (n_lectores >= 1))
  {
           n_escritores_esperando++;
           printf("Escritor %d espera\n",n_escritor);
           pthread_cond_wait(&se_puede_acceder, &mutex);
           n_escritores_esperando--;
  }
  n_escritores++;
  printf("Escritor %d va a escribir\n",n_escritor);

  sleep(random()%3); /* Simula acceso lectura fichero */

  printf("Escritor %d deja de escribir\n",n_escritor);
  n_escritores--;
  pthread_cond_broadcast(&se_puede_acceder);
  pthread_mutex_unlock(&mutex);
```

```
  pthread_exit(0);
}

int main()
{ int i, *v;
  pthread_t *hilos_lectores, *hilos_escritores;

  time_t t;
  srandom(time(&t));

  hilos_lectores   = malloc(N_HILOS * sizeof(pthread_t));
  hilos_escritores = malloc(N_HILOS * sizeof(pthread_t));
  v                = malloc(N_HILOS * sizeof(int));

  pthread_mutex_init(&mutex, NULL);
  pthread_cond_init(&se_puede_acceder, NULL);

  for (i = 0; i < N_HILOS; i++)
  {   v[i] = i;
      pthread_create(&hilos_lectores[i], NULL, f_lector, (void *)&v[i]);
  }
  for (i = 0; i < N_HILOS; i++)
      pthread_create(&hilos_escritores[i], NULL, f_escritor, (void *)&v[i]);
  for (i = 0; i < N_HILOS; i++)
      pthread_join(hilos_lectores[i], NULL);
  for (i = 0; i < N_HILOS; i++)
      pthread_join(hilos_escritores[i], NULL);

  pthread_mutex_destroy(&mutex);
  pthread_cond_destroy(&se_puede_acceder);

  pthread_exit(0);
}
```

En esta soluci□n, si acaba de leer un hilo y hay hilos escritores esperando se desbloquea a todos los hilos bloqueados en la variable de condici□n. Si estaban bloqueados hilos lectores en esa variable de condici□n se desbloquear□n aunque, mientras haya escritores esperando, se volver□n a bloquear ya que los escritores tiene mayor prioridad para acceder al fichero. Por otro lado, cuando acaba de escribir un hilo en el fichero desbloquea a todos los hilos que est□n esperando para acceder al fichero. Al igual que antes, si hay escritores y lectores esperando, los lectores se tendr□n que volver a bloquear hasta que no haya escritores. Una forma de evitar estos bloqueos innecesarios de los hilos lectores y, consecuentemente, de mejorar la eficiencia de esta soluci□n, ser□a utilizar dos variables de condici□n, una para los hilos lectores y otra para los hilos escritores.

Soluci□n II (con dos variables de condici□n):

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define N_HILOS  5

int n_lectores=0, n_escritores=0, n_escritores_esperando=0;
pthread_mutex_t mutex;
pthread_cond_t acceder_lectores, acceder_escritores;

void *f_lector (void *indice)
{ int i, n_lector, resultado;
  n_lector = *(int *)indice;
  sleep(random()%5);

  pthread_mutex_lock(&mutex);
  printf("Lector %d llega\n",n_lector);
  while ((n_escritores == 1) || (n_escritores_esperando>0))
```

```
  {
          printf("Lector %d espera\n",n_lector);
          pthread_cond_wait(&acceder_lectores, &mutex);
  }
  n_lectores++;
  printf("Lector %d va a leer\n",n_lector);
  pthread_mutex_unlock(&mutex);

  sleep(random()%3); /* Simula acceso lectura fichero */

  pthread_mutex_lock(&mutex);
  printf("Lector %d deja de leer\n",n_lector);
  n_lectores--;
  if (n_escritores_esperando>0)  pthread_cond_signal(&acceder_escritores);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}

void *f_escritor (void *indice)
{ int i, n_escritor, resultado;
  n_escritor= *(int *)indice;
  sleep(random()%3);

  pthread_mutex_lock(&mutex);
  printf("Escritor %d llega\n",n_escritor);
  while ((n_escritores == 1) || (n_lectores >= 1))
  {
          n_escritores_esperando++;
          printf("Escritor %d espera. nlectores=%d ", n_escritor, n_lectores);
          printf("nescritores=%d n_escritores_esperando=%d\n", n_escritores,
n_escritores_esperando);
          pthread_cond_wait(&acceder_escritores, &mutex);
          n_escritores_esperando--;
  }
  n_escritores++;
  printf("Escritor %d va a escribir\n",n_escritor);

  sleep(random()%3); /* Simula acceso lectura fichero */

  printf("Escritor %d deja de escribir\n",n_escritor);
  n_escritores--;
  if (n_escritores_esperando>0) pthread_cond_signal(&acceder_escritores);
  else                          pthread_cond_broadcast(&acceder_lectores);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}

int main()
{ int i, *v;
  pthread_t *hilos_lectores, *hilos_escritores;

  time_t t;
  srandom(time(&t));

  hilos_lectores =   malloc(N_HILOS * sizeof(pthread_t));
  hilos_escritores = malloc(N_HILOS * sizeof(pthread_t));
  v                = malloc(N_HILOS * sizeof(int));

  pthread_mutex_init(&mutex, NULL);
  pthread_cond_init(&acceder_lectores, NULL);
  pthread_cond_init(&acceder_escritores, NULL);

  for (i = 0; i < N_HILOS; i++)
  {   v[i] = i;
      pthread_create(&hilos_lectores[i], NULL, f_lector, (void *)&v[i]);
  }
  for (i = 0; i < N_HILOS; i++)
      pthread_create(&hilos_escritores[i], NULL, f_escritor, (void *)&v[i]);
  for (i = 0; i < N_HILOS; i++)
```

```
      pthread_join(hilos_lectores[i], NULL);
  for (i = 0; i < N_HILOS; i++)
      pthread_join(hilos_escritores[i], NULL);

  pthread_mutex_destroy(&mutex);
  pthread_cond_destroy(&acceder_lectores);
  pthread_cond_destroy(&acceder_escritores);

  pthread_exit(0);
}
```

En las dos implementaciones anteriores se podr�a prescindir de la variable n_escritores y de la condici�n que se hace sobre ella en el bucle while de las funciones f_lector y f_escritor. Pensemos que esta variable solo obtiene el valor 1 cuando hay un escritor accediendo a los datos y, en esa situaci�n, ni un lector ni otro escritor pueden acceder al mutex porque lo tiene el escritor que est� accediendo a los datos. La implementaci�n resultante para la segunda soluci�n propuesta quedar�a, por tanto, de la siguiente manera:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#define N_HILOS 5

int n_lectores=0, n_escritores_esperando=0;
pthread_mutex_t mutex;
pthread_cond_t acceder_lectores, acceder_escritores;


void *f_lector (void *indice) {
  int i, n_lector, resultado;
  n_lector = *(int *)indice;
  sleep(random()%5);
  pthread_mutex_lock(&mutex);
  while (n_escritores_esperando>0){
    printf("Lector %d espera\n",n_lector);
    pthread_cond_wait(&acceder_lectores, &mutex);
  }
  n_lectores++;
  printf("Lector %d va a leer\n",n_lector);
  pthread_mutex_unlock(&mutex);

  sleep(random()%3); /* Simula acceso lectura fichero */

  pthread_mutex_lock(&mutex);
  printf("Lector %d deja de leer\n",n_lector);
  n_lectores--;
  if (n_escritores_esperando>0)  pthread_cond_signal(&acceder_escritores);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}

void *f_escritor (void *indice) {
  int i, n_escritor, resultado;
  n_escritor= *(int *)indice;
  sleep(random()%3);

  pthread_mutex_lock(&mutex);
  printf("Escritor %d llega\n",n_escritor);
  while (n_lectores > 0 ){
    n_escritores_esperando++;
    printf("Escritor %d espera. nlectores=%d ", n_escritor, n_lectores);
    printf("n_escritores_esperando=%d\n", n_escritores_esperando);
    pthread_cond_wait(&acceder_escritores, &mutex);
    n_escritores_esperando--;
  }
  printf("Escritor %d va a escribir \n",n_escritor);
  sleep(random()%3); /* Simula acceso lectura fichero */
  printf("Escritor %d deja de escribir \n",n_escritor);
  if (n_escritores_esperando>0)
```

```
    pthread_cond_signal(&acceder_escritores);
  else
    pthread_cond_broadcast(&acceder_lectores);
  pthread_mutex_unlock(&mutex);
  pthread_exit(0);
}


int main() {
  int i, n=0, *v;
  pthread_t *hilos_lectores, *hilos_escritores;
  time_t t;
  srandom(time(&t));
  hilos_lectores   =malloc(N_HILOS * sizeof(pthread_t));
  hilos_escritores = malloc(N_HILOS * sizeof(pthread_t));
  v                = malloc(N_HILOS * sizeof(int));

  pthread_mutex_init(&mutex, NULL);
  pthread_cond_init(&acceder_lectores, NULL);
  pthread_cond_init(&acceder_escritores, NULL);


  for (i = 0; i < N_HILOS; i++)
  {   v[i] = i;
      pthread_create(&hilos_lectores[i], NULL, f_lector, (void *)&v[i]);
  }
  for (i = 0; i < N_HILOS; i++)
    pthread_create(&hilos_escritores[i], NULL, f_escritor, (void *)v[i]);
  for (i = 0; i < N_HILOS; i++)
    pthread_join(hilos_lectores[i], NULL);
  for (i = 0; i < N_HILOS; i++)
    pthread_join(hilos_escritores[i], NULL);
  pthread_exit(0);
}


*****************
Examen enero 2016
*****************

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define N_HILOS  3

int n_comiendo=0, n_pensando=0;
pthread_mutex_t mutex;
pthread_cond_t se_pueda_comer, se_pueda_pensar;


void *f_filosofo(void *indice)
{ int i, n_filosofo;
  n_filosofo= *(int *)indice;
  sleep(random()%5);
  printf("Fil�sofo %d llega\n",n_filosofo);

  while (1)
  {
     pthread_mutex_lock(&mutex);
     while (n_pensando > 0)
     {
           printf("Filosofo %d esperando para comer\n",n_filosofo);
           pthread_cond_wait(&se_pueda_comer, &mutex);
     }
     n_comiendo++;
```

```
      printf("Filosofo %d va a comer\n",n_filosofo);
      pthread_mutex_unlock(&mutex);

      sleep(random()%3); /* Simula que come */

      pthread_mutex_lock(&mutex);
      printf("Filosofo %d deja de comer\n",n_filosofo);
      n_comiendo--;
      if (n_comiendo== 0) pthread_cond_broadcast(&se_pueda_pensar);
      while (n_comiendo > 0)
      {
            printf("Filosofo %d esperando para pensar\n",n_filosofo);
            pthread_cond_wait(&se_pueda_pensar, &mutex);
      }
      n_pensando++;
      printf("Filosofo %d va a pensar\n",n_filosofo);
      pthread_mutex_unlock(&mutex);

      sleep(random()%3); /* Simula que piensa */

      pthread_mutex_lock(&mutex);
      printf("Filosofo %d deja de pensar\n",n_filosofo);
      n_pensando--;
      if (n_pensando== 0) pthread_cond_broadcast(&se_pueda_comer);
      pthread_mutex_unlock(&mutex);
  }
  pthread_exit(0);
}

int main()
{ int i, *v;
  pthread_t *hilos_filosofos;

  time_t t;
  srandom(time(&t));

  hilos_filosofos =   malloc(N_HILOS * sizeof(pthread_t));
  v               =   malloc(N_HILOS * sizeof(v));

  pthread_mutex_init(&mutex, NULL);
  pthread_cond_init(&se_pueda_pensar, NULL);
  pthread_cond_init(&se_pueda_comer, NULL);

  for (i = 0; i < N_HILOS; i++)
  {   v[i] = i;
      pthread_create(&hilos_filosofos[i], NULL, f_filosofo, (void *)&v[i]);
  }
  for (i = 0; i < N_HILOS; i++)
      pthread_join(hilos_filosofos[i], NULL);

  pthread_mutex_destroy(&mutex);
  pthread_cond_destroy(&se_pueda_pensar);
  pthread_cond_destroy(&se_pueda_comer);

  pthread_exit(0);
}
```

Cabe indicar en primer lugar que cada fil�sofo ha de realizar la acci�n de comer y la de pensar en la misma funci�n, tal y como se indicaba en el enunciado. Tanto la acci�n de comer como la de pensar ha de realizarse fuera del mutex para que varios hilos puedan comer a la vez y pensar a la vez. Adem�s, el acceso a las variables globales accesibles por todos los hilos ha de realizarse dentro del mutex. Y, finalmente, si se elimina la condici�n que precede a la funci�n pthread_cond_broadcast y se ejecuta esta funci�n de manera incondicional, el programa ser�a m�s ineficiente ya que se podr�an desbloquear hilos que han de volver a bloquearse inmediatamente porque no cumplen las condiciones necesarias para continuar su ejecuci�n.